

平成 2 5 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 8 月期）

試験問題 基礎科目（情報基礎）

Fundamentals of Computer Science

[注意事項][Instructions]

1. 試験開始の合図があるまで、問題の中を見てはいけません。
Do NOT open this booklet before the examination starts.
2. 全ての解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
Fill in the designated spaces on each answer sheet with the name of the graduate school, name of your main (major) field, and the examination number.
3. この問題は全部で 10 ページ（表紙を除く）です。1～5 ページは日本語版、6～10 ページは英語版です。
This booklet consists of 10 pages, excluding the cover sheet. The Japanese version is shown on pages 1-5 and the English version on pages 6-10.
4. 解答用紙（罫線有り）を 2 枚、下書き用紙（白紙）を 1 枚配布します。
You are given two ruled answer sheets and one white draft sheet.
5. 問題は全部で 2 問あります。問題 I の解答を 1 枚目、問題 II の解答を 2 枚目の解答用紙に、必ず分けて記入すること。
There are two problems. Write the answers to Problem I on the first answer sheet and the answers to Problem II on the second answer sheet.
6. 解答用紙に解答を記述する際に、問題 I の解答か問題 II の解答かを必ず明記すること。
When writing the answers, clearly label the problem number on each answer sheet.

平成 2 4 年 8 月 2 3 日

問題 I の解答は 1 枚目の解答用紙に記入すること。問題 I の解答であることを明記すること。

問題 I 文字列を照合する C 言語のプログラムを考える。文字列を照合するとは、テキストと呼ばれる文字列の中に、パターンと呼ばれる文字列に一致する部分があるかどうかを調べる問題である。そのような部分が見つかったら、テキストにおけるその部分の先頭位置を答えとする。テキストに複数のパターンが含まれる場合は、最初に現れるものを調べる。図 1 では、 n 文字のテキストの文字列 `text` の中に、 m 文字のパターンの文字列 `pattern` が含まれていることを表しており、答えは p となる。ただし、データの末尾には、null 文字 `'\0'` が置かれている。また以下の設問では、テキストおよびパターンは、ともに空文字列ではないものとする。

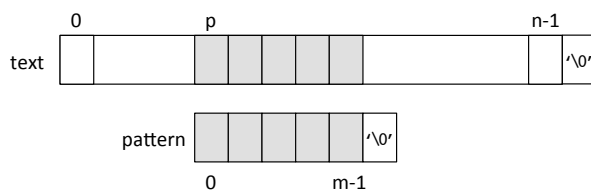


図 1 C 言語における文字列の照合

(1) 図 1 に示したようにテキストとパターンが一致する時、以下の条件が成立する。以下の空欄 (A), (B), (C) に、該当する添字を埋めなさい。

```
pattern[0]    ==  text[ (A) ]
pattern[1]    ==  text[ (B) ]
...
pattern[m-1]  ==  text[ (C) ]
```

(2) 文字列の照合をもっとも単純に行う方法は、テキストにパターンを重ね、パターンの先頭から 1 文字ずつ照合し一致するかどうかを調べ、一致しなければ重ねる位置を 1 文字ずらす方法である。以下の関数は、ASCII コードの文字列を対象に、この方法を実装したものである。この関数はパターンの照合が成功した場合にはテキストにおいて一致した部分の先頭位置を返し、失敗した場合には -1 を返す。空欄 (D), (E) を埋めて、プログラムを完成させなさい。

```
int simple_match(char *text, char *pattern)
{
    int i = 0, j = 0;
    while (text[i] != '\0') {
        if (text[i] != pattern[j]) {
            (D) ;
            j = 0;
        } else if (pattern[j + 1] == '\0')
            return (E) ;
        else
            i++, j++;
    }
    return -1;
}
```

(3) 文字列照合をより高速に行うために、パターン最後の文字から先頭に向かって照合し、照合に失敗した場合、失敗した文字とパターンに含まれる文字の情報を用いて可能な限り右に大きくずらす、という方法を考える。図2に、照合に失敗した場合の、パターンのずらし方を示す。図2(a)では、テキストの x と、パターンの b との照合に失敗している。照合に失敗したテキストの x がパターンに含まれないため、テキストの照合に失敗した文字の次の文字まで、パターンをずらすことができる。図2(b)では、テキストの a とパターンの c との照合に失敗している。この場合は、照合に失敗したテキストの a がパターンに含まれるので、照合に失敗したテキストの a と、パターンの a をあわせるように、パターンをずらす。

パターンをずらす文字数は予め、パターンの文字ごとに計算し、配列に格納する。この配列を、ここでは skip[] とする。skip[] には、文字コードの値を添字として、その文字が最後尾から何文字目に現れるかが入る。最後尾の文字に対応する skip[] の要素は 0 になり、パターンに現れない文字に対応する要素はパターンの文字数になる。例えば、図2のパターンでは、skip['a'] には 2 が入る。

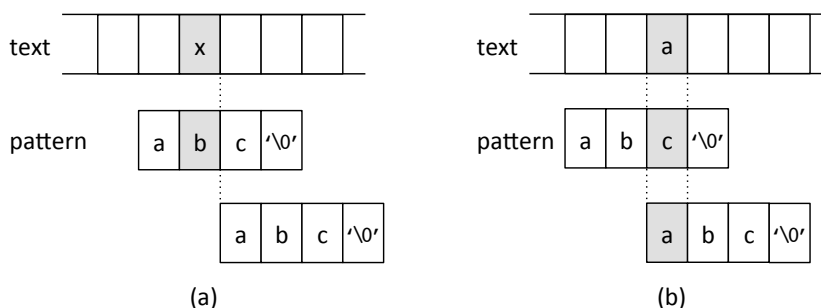


図2 より高速な文字列の照合

以下の関数は、ASCII コードの文字列を対象に、上記の方法を実装したものである。3 番目の引数 n は text の文字数 (strlen(text) の値) であり、戻り値は simple_match() と同じである。また、calc_skip() は、与えられた pattern に対して配列 skip[] の要素を設定し、pattern に含まれる文字数を返すものとする。空欄 (F), (G) を埋めてプログラムを完成させなさい。

```
#define NUM_CHAR 128
int skip[NUM_CHAR];
int faster_match(char *text, char *pattern, int n)
{
    int m = calc_skip(pattern) - 1;
    int i = m, j;
    while (i < n) {
        j = m;
        while (j >= 0) {
            if (text[i] == pattern[j]) {
                if (j == 0)
                    return (F);
                i--, j--;
            } else
                break;
        }
        (G);
    }
    return -1;
}
```

(4) 次の `text`, `pattern`, `n` を引数として, 設問 (3) のプログラムを呼び出した時, 照合が成功するまでに行われる文字照合の回数を答えなさい.

```
char *text = "XYXZde0XZZkWXYZ";
char *pattern = "WXYZ";
int n = strlen(text);
```

(5) 以下は, 設問 (3) における `calc_skip()` のプログラムである. 空欄 (H), (I) を埋めて完成させなさい.

```
int calc_skip(char *pattern)
{
    int i, m;
    m = strlen(pattern);
    for (i = 0; i < NUM_CHAR; i++)
        skip[i] = m;
    for (i = 0; i < m; i++)
        (H) ;
    return (I) ;
}
```

問題 II の解答は 2 枚目の解答用紙に記入すること、問題 II の解答であることを明記すること。

問題 II

(1) 図 1 に示した根付き木 T_1 について答えなさい。なお、二つの節点が枝で接続されているとき、上の節点を親とする。

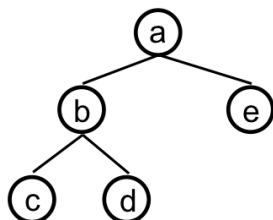


図 1 根付き木 T_1

- 根付き木は、 (V, E, r) によって表わされる。ここで V は空でない節点集合、 $E \subseteq V \times V$ は枝集合、 $r \in V$ は根節点である。 T_1 に対して V, E, r を示しなさい。
- 根付き木から葉を取り除く操作を繰り返すことによって部分木を得ることができる。このようにして得られる T_1 の部分木は全部で何通りあるか答えなさい。なお、 T_1 そのものも T_1 の部分木であるとする。
- 根付き木 T, T' に対して、 T' が T の部分木であることを $T' \preceq T$ と表記する。また、 T の全ての部分木の集合を $\mathcal{T}$ とする。 $(\mathcal{T}, \preceq)$ は 1) 反射律, 2) 反対称律, 3) 推移律のそれぞれを満たすかどうかを答えなさい。満たす場合は理由を、満たさない場合は反例を示すこと。

(2) 図 2 に示した根付き順序木 T_2 について答えなさい。なお、各節点は前置順で順序付けられているものとし、根節点の前置順は 0 である。また、二つの節点が枝で接続されているとき、上の節点を親とする。

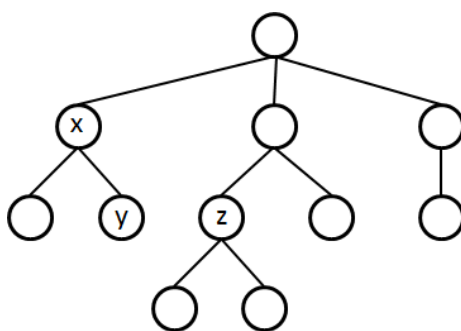


図 2 根付き順序木 T_2

- 各節点に固有の整数値 (ラベル) を与え、ラベル同士を比較することで、節点間の関係、例えば親子、先祖子孫などを判別できるようにしたい。ここで、前置順で i 番目の節点 v_i に対して i 番目の素数 p_i を対応付ける。このとき、 v_i のラベル l_{v_i} を以下のように定義する。

$$l_{v_i} = \begin{cases} 1 & (i = 0) \\ l_{\text{parent}(v_i)} \cdot p_i & (i = 1, 2, \dots) \end{cases}$$

ここで, $l_{parent(v_i)}$ は v_i の親ノードのラベルである. 図中 x, y, z の各節点について, 1) 前置順 i , 2) 素数 p_i , 3) ラベル l_{v_i} を列挙しなさい. なお, i と p_i の対応は以下の表の通りである.

i	1	2	3	4	5	6	7	8	9	10
p_i	2	3	5	7	11	13	17	19	23	29

- (b) 二つの異なるノード v_a, v_d について, l_d が l_a で割り切れるときかつそのときのみ, v_a が v_d の先祖ノードであることを示しなさい.
- (c) 二つの異なるノード v_a, v_b について, 以下の処理を行なう方法を, ラベルを用いて記述しなさい.
1. v_a が v_b の親であるかどうかを判別する.
 2. v_a と v_b が兄弟ノードであるかどうかを判別する.
 3. v_a, v_b が根以外の共通祖先を持つかどうかを判別する.
 4. v_a, v_b の最小共通祖先 (Least Common Ancestor) のラベルを求める. ここで最小共通祖先とは, 共通の祖先ノードのうち最も葉に近いものを指す.

Write the answers to Problem I on the first answer sheet, and clearly label it at the top of the page as “Problem I.”

Problem I Consider a C language program that performs a substring search. Substring search is a procedure that, given a *text* string and a *pattern* string, examines if there is an occurrence of the pattern within the text. If any occurrences are found, it outputs the position of the first character of the first occurrence within the text. Figure 1 shows that the n -character text string contains a substring that matches the m -character pattern string, and that the output is p . A string ends with a null character, `'\0'`. In the following questions, both the text and the pattern are non-null strings.

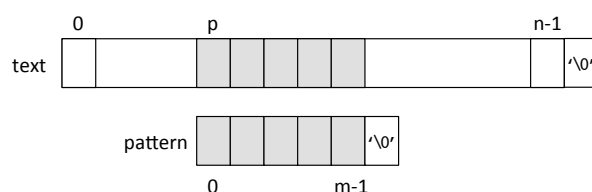


Figure 1. Substring search problem in the C language.

(1) When the text and the pattern match as shown in Figure 1, the following equations hold. In the following equations, fill the indexes in the boxes (A), (B), and (C) to complete the equations.

```

pattern[0]    ==  text[ (A) ]
pattern[1]    ==  text[ (B) ]
...
pattern[m-1]  ==  text[ (C) ]

```

(2) The simplest method for substring search is to check, for each possible position in the text, if the all characters in the text and the pattern match. If there is a character that does not match, the pattern is moved one character right. The following function implements this method taking the text and pattern of ASCII characters as its arguments. This function returns the position of the first character of the pattern’s first occurrence within the text if the search succeeds. Otherwise, it returns `-1`. Fill in the boxes (D) and (E) to complete the program.


```

int simple_match(char *text, char *pattern)
{
    int i = 0, j = 0;
    while (text[i] != '\0') {
        if (text[i] != pattern[j]) {
            (D) ;
            j = 0;
        } else if (pattern[j + 1] == '\0')
            return (E) ;
        else
            i++, j++;
    }
    return -1;
}

```

(3) A faster substring search is possible by the following method. This method scans the pattern from right to left when trying to match it against the text. It also takes advantage of the information contained in the characters of the pattern and tries to shift the pattern by as many characters as possible if the matching fails. Figure 2 (a) and (b) show two ways to shift the pattern if the matching fails. Figure 2 (a) shows that the character **x** in the text and the character **b** in the pattern failed to match. Since the pattern does not contain the character **x**, which failed to match, in the text, the pattern is shifted to the next to the character that failed to match. Figure 2 (b) shows that the character **a** in the text and the character **c** in the pattern failed to match. Since the pattern contains the character **a**, which failed to match, in the text, the pattern is shifted to match the character **a** in the text and the pattern.

The number of characters to shift is calculated in advance for each character in the pattern and is stored in an array. Let the array be `skip[]` in this problem. An element in `skip[]` is indexed by a character code, and tells where the character of the code appears in the pattern counting from its end. The element for the last character is 0, and the elements for the characters that are not in the pattern are the number of the characters in the pattern. For example, for the following figure's pattern, `skip['a']` contains 2.

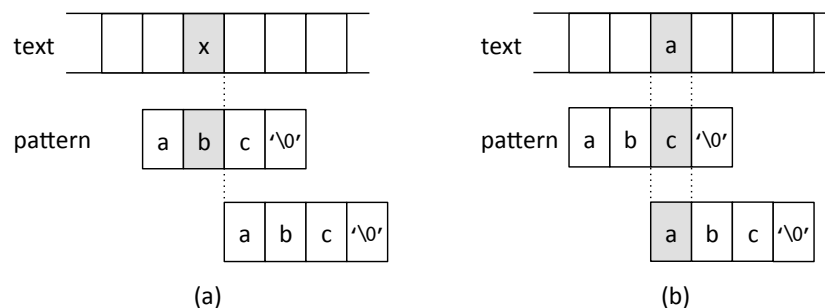


Figure 2. Faster substring search.

The following function implements the above method, taking the text and pattern of ASCII characters as its argument. The third argument `n` is the number of characters (the value of `strlen(text)`) contained in the text. The return value is the same as the `simple_match()`. `calc_skip()` calculates the elements of `skip[]` for the given `pattern`, and returns the number of the characters contained in `pattern`. Fill in the boxes (F) and (G) to complete the program.

```
#define NUM_CHAR 128
int skip[NUM_CHAR];
int faster_match(char *text, char *pattern, int n)
{
    int m = calc_skip(pattern) - 1;
    int i = m, j;
    while (i < n) {
        j = m;
        while (j >= 0) {
            if (text[i] == pattern[j]) {
                if (j == 0)
                    return (F) ;
                i--, j--;
            } else
                break;
        }
        (G) ;
    }
    return -1;
}
```

(4) If the following `text`, `pattern`, and `n` are inputted to the above program of Question (3), find the number of character matching, performed until substring search succeeds.

```
char *text = "XYXZde0XZZkWXYZ";
char *pattern = "WXYZ";
int n = strlen(text);
```

(5) The following is the program of `calc_skip()` of Question (3). Fill in the boxes (H) and (I) to complete the program.

```
int calc_skip(char *pattern)
{
    int i, m;
    m = strlen(pattern);
    for (i = 0; i < NUM_CHAR; i++)
        skip[i] = m;
    for (i = 0; i < m; i++)
        (H) ;
    return (I) ;
}
```

Write the answers to Problem II on the second sheet, and clearly label it at the top of the page as “Problem II.”

Problem II

(1) Answer the following problems regarding the rooted tree T_1 in Figure 1. Note that, for a pair of connected nodes, the upper node is the parent.

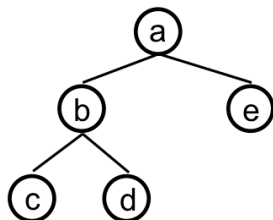


Figure 1. A rooted tree T_1 .

- (a) A rooted tree is denoted by (V, E, r) , where $V, E \subseteq V \times V$, and $r \in V$ are the non-empty set of nodes, the set of edges, and the root node, respectively. Enumerate the elements of V and E , and show r for T_1 .
- (b) For a rooted tree, a subtree can be obtained by eliminating one of its leaf nodes. Write the number of possible subtrees by applying this operation repeatedly. Note that T_1 itself is regarded as one of its subtrees.
- (c) Given two rooted trees T and T' , let $T' \preceq T$ denote that T' is a subtree of T . Also let $\mathcal{T}$ be the set of all subtrees of T . For each of 1) the reflexive law, 2) the antisymmetric law, and 3) the transitive law, decide whether $(\mathcal{T}, \preceq)$ follows it or not. If true, give the reason; otherwise, give a counterexample.

(2) Answer the following questions regarding the rooted ordered tree T_2 in Figure 2. Note that the nodes are ordered according to the preorder traversal, and let the root's order be 0. Also note that, for a pair of connected nodes, the upper node is the parent.

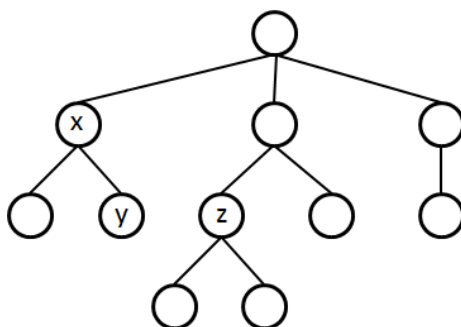


Figure 2. A rooted ordered tree T_2 .

- (a) We assign an integer value (hereafter called a “label”) to each node by which we can identify the relationship between two nodes, such as parent-child and ancestor-descendant, by the labels. Now, we associate the i -th node v_i with the i -th prime number p_i . Then,

the label l_{v_i} of v_i can be defined as follows:

$$l_{v_i} = \begin{cases} 1 & (i = 0) \\ l_{parent(v_i)} \cdot p_i & (i = 1, 2, \dots) \end{cases}$$

where $l_{parent(v_i)}$ is the label of v_i 's parent. For each node labeled by x, y, or z in Figure 2, write 1) the preorder number i , 2) the associated prime number p_i , and 3) the label l_{v_i} . The table below shows the correspondence between i and p_i .

i	1	2	3	4	5	6	7	8	9	10
p_i	2	3	5	7	11	13	17	19	23	29

- (b) Let v_a and v_d be two distinct nodes. Prove that v_a is an ancestor of v_d if and only if l_d is divisible by l_a .
- (c) Let v_a and v_b be two distinct nodes. Describe the procedures that achieve the following processes using the labels.
1. Detect whether v_a is the v_b 's parent.
 2. Detect whether v_a and v_b are siblings.
 3. Detect whether v_a and v_b have a common ancestor other than the root.
 4. Compute the label of the least common ancestor of v_a and v_b . Note that, given a set of nodes, the least common ancestor of the nodes is the common ancestor which is closest to the leaf nodes.